

BILL MANAGEMENT SYSTEM

Project README / Reference Document

Bill Management System

A web-based Bill Management System developed using Flask, SQLite, HTML5, CSS3, Bootstrap 5, and JavaScript. The application helps businesses create customer bills, track item and quantity details, calculate bill amounts, manage payment status, and view outstanding dues through a simple and professional dashboard.

Features

- Bill Creation and Management
- Customer Details on Bill
- Item and Quantity Entry
- Automatic Amount Handling
- Payment Mode Selection (Cash / UPI / Card)
- Paid and Unpaid Status Tracking
- Outstanding Amount Calculation
- Bill Search
- Dashboard Statistics
- Bill Summary View
- Delete Bill Record
- Responsive Bootstrap 5 Interface
- Professional Dashboard Design
- SQLite Database Storage

Tech Stack

Frontend

- HTML5
- CSS3
- Bootstrap 5
- JavaScript

Backend

- Python
- Flask

Database

- SQLite

Project Structure

```
Bill-Management-System/  
■■ app.py  
■■ requirements.txt  
■■ README.md  
■■ database.db  
■■  
■■ static/  
■ ■■ style.css  
■ ■■ js/  
■ ■■ script.js  
■■  
■■ templates/  
■■ base.html  
■■ index.html  
■■ bills.html  
■■ add_bill.html  
■■ bill_details.html
```

Database Table

Bill Table

Field — Description

Bill ID — Unique bill identifier

Customer Name — Name of the customer

Phone — Customer contact number

Item Description — Description of billed item/service

Quantity — Quantity of item

Amount — Total bill amount

Bill Date — Date the bill was created

Due Date — Payment due date

Payment Mode — Cash / UPI / Card

Status — Paid / Unpaid

Application Workflow

- 1. Open Dashboard
- 2. Click Add Bill
- 3. Enter Customer Details
- 4. Enter Item and Quantity
- 5. Enter Bill Amount
- 6. Select Payment Mode
- 7. Set Bill and Due Date
- 8. Save Bill
- 9. View Bill in Bills List
- 10. Search Bill by Customer or Item
- 11. Open Bill Details
- 12. Mark Bill as Paid
- 13. Delete Bill if Needed

- 14. View Dashboard Summary

API Integration

An external API can optionally be integrated into the project. For example, a public currency conversion, SMS notification, or payment gateway API can be used to demonstrate REST API integration.

User Input → Flask Route → API Request → JSON Response → Process API Data → Display Result

If API integration is included, the requests library can be used:

```
pip install requests
```

The API should be used only as an additional feature. The core Bill Management System can work using Flask and SQLite.

User Interface

- Navigation Bar
- Dashboard Cards
- Bill Form
- Bills Table
- Search Box
- Status Badges
- Bill Details View
- Summary Sections

Installation

1. Clone or Download the Project

```
git clone <your-repository-url>  
cd Bill-Management-System
```

2. Create Virtual Environment

```
python -m venv venv
```

3. Activate Virtual Environment

```
Windows: venv\Scripts\activate  
Linux / macOS: source venv/bin/activate
```

Install Dependencies

```
pip install -r requirements.txt
```

Example requirements.txt:

```
Flask
```

Run the Application

```
python app.py
```

Open the application in your browser:

<http://127.0.0.1:5000/>

Screenshots

Add your actual project screenshots here:

Dashboard: ![Dashboard](screenshots/dashboard.png)

Bills List: ![Bills List](screenshots/bills.png)

Add Bill: ![Add Bill](screenshots/add_bill.png)

Bill Details: ![Bill Details](screenshots/bill_details.png)

Future Enhancements

- Login and User Authentication
- Admin and Staff Roles
- GST / Tax Calculation
- SMS Notifications
- Email Notifications
- PDF Bill/Invoice Generation
- Payment Gateway Integration

Learning Outcomes

- Python programming
- Flask web application development
- HTML and CSS
- Bootstrap 5
- JavaScript
- Basic REST API integration concepts

Author

Student Name: Rohan Kumar

USN: U18IN24S0038

Course: BCA

Project: Bill Management System

Technologies: Python | Flask | SQLite | HTML | CSS | Bootstrap | JavaScript | REST API

GitHub

<https://github.com/your-username/bill-management-system>

License

This project is developed for educational, academic, internship, and portfolio purposes.